Porovnanie MLP a CNN pri rozpoznávaní rukou písaných číslic

Obsah

Úloha 7: Porovnanie MLP a CNN **2**
Cieľ úlohy . . . 2
Definícia úlohy . . . 2
Postup riešenia . . . 2

Odovzdanie a hodnotenie **4**
Dokumentácia výsledkov (PDF) . . . 4
Obhajoba riešenia . . . 4

Odporúčané nástroje a technická realizácia **6**
Matlab . . . 6
Python . . . 6

Teoretické poznámky **7**
MNIST a základné princípy rozpoznávania číslic . . . 7
MLP vs. CNN a reprezentácia vstupu . . . 8
Rozdelenie dát . . . 8
Férové porovnávanie architektúr . . . 8
Ako interpretovať výsledky pri klasifikácii obrázkov . . . 9

Ukážky minimálneho rozsahu tabuliek v dokumentácii **10**

Úloha 7: Porovnanie MLP a CNN pri rozpoznávaní rukou písaných číslic (7b)

Cieľ úlohy

Vytvorte a natrénujte dve neurónové siete pre rozpoznávanie rukou písaných číslic: **viacvrstvovú perceptrónovú sieť (MLP)** a **konvolučnú neurónovú sieť (CNN)**. Cieľom je porovnať klasický ANN prístup a deep learning prístup na rovnakom datasete.

Pre **MLP** musí mať **najlepšia z porovnávaných štruktúr** na testovacích dátach **priemernú úspešnosť aspoň 97%**, pričom priemer sa určí z **5 nezávislých spustení trénovania odznova**. Pre **CNN** musí mať **najlepšia z porovnávaných štruktúr** na testovacích dátach **priemernú úspešnosť aspoň 99%**, opäť ako priemer z **5 nezávislých spustení trénovania odznova**.

Definícia úlohy

Na riešenie použite dataset **MNIST** s rukou písanými číslicami 0 až 9. Dataset je už rozdelený na **60 000 tréningových** a **10 000 testovacích** vzoriek (približne **85.7% train** a **14.3% test**). Testovacia časť zostáva pevná, prípadnú validačnú množinu vytvorte ďalším rozdelením tréningovej časti.

Oba prístupy, **Matlab aj Python**, používajú ten istý dataset **MNIST**. Dataset pre Matlab aj Python je dostupný v repozitári **UMINT-UNS_data**. Pre Matlab použite súbor `MNIST_MATLAB.zip`, pre Python použite oficiálny dataset `MNIST`, ktorý sa načítava priamo z knižnice `torchvision`.

Postup riešenia

1. Použite dataset **MNIST**. Testovaciu časť ponechajte oddelenú na finálne vyhodnotenie, prípadnú validačnú množinu vytvorte ďalším rozdelením tréningových dát.

2. Pri riešení v **Matlabe aj Pythone** používajte ako základ ukážkové kódy zo **Zadania 5** a tie ďalej upravujte pre potreby tejto úlohy. **AI slop** alebo iný cudzí kód sa považuje za **plagiát**.

3. Navrhnite a porovnajte **aspoň 2 rôzne štruktúry MLP**. Každú z nich **spustite minimálne 5-krát odznova** a pre každú štruktúru vyčíslite minimálnu, maximálnu a priemernú úspešnosť klasifikácie na trénovacích a testovacích dátach. Pre MLP časť tak ide **celkovo aspoň o 10 spustení**.

4. Navrhnite a porovnajte **aspoň 3 rôzne architektúry CNN**. Každú z nich **spustite minimálne 5-krát odznova** a pre každú štruktúru vyčíslite minimálnu, maximálnu a priemernú úspešnosť klasifikácie na trénovacích a testovacích dátach. Pre CNN časť tak ide **celkovo aspoň o 15 spustení**.

5. Dodatočne porovnajte **aspoň 3 modely s rôznym nastavením dropout-u** a sledujte, aký má dropout vplyv na pretrénovanie a na rozdiel medzi tréningovou a validačnou chybou. Každý z týchto modelov **spustite minimálne 5-krát odznova**. Táto časť preto predstavuje **ďalších aspoň 15 spustení**.

6. Pri porovnávaní jednotlivých štruktúr zachovajte **rovnaké hlavné hyperparametre** trénovania, learning rate, batch size a ostatné hlavné nastavenia. Výnimkou môže byť iba **early stopping** a počet epoch (pri dropout).

7. Pre **najlepší výsledok z každej porovnávanej štruktúry** zdokumentujte priebeh učenia, výslednú kontingenčnú maticu a dosiahnuté úspešnosti klasifikácie. Následne vykonajte **porovnanie MLP a CNN**. Porovnajte najmä priebehy trénovania, chybu a úspešnosť klasifikácie.

8. Aspoň pri **jednej vybranej MLP sieti** a **jednej vybranej CNN sieti** zmerajte rozdiel času trénovania pri použití **CPU** a **GPU** a výsledky stručne porovnajte.

9. Pre najlepšie MLP aj CNN otestujte z každého čísla aspoň jednu vzorku. Vybrané vzorky prezentujte **vizuálne** ako obrázky číslic s uvedením skutočnej triedy, predikovanej triedy a výstupu siete.

Poznámka: Pri porovnávaní MLP a CNN je cieľom porovnávať **porovnateľné nastavenia**. Ak sa mení architektúra siete, ostatné hlavné hyperparametre by mali zostať čo najviac rovnaké, aby bolo možné výsledky medzi modelmi férovo porovnať. Ak používate validačné dáta, graf **loss vs. epoch** zobrazujte ako spoločný graf trénovacej a validačnej chyby. Na testovacích dátach sa potom uvádzajú už len výsledné metriky po natrénovaní siete.

Zároveň výsledky musia v dokumentácii spracované nielen vo forme grafov, ale aj v **prehľadných tabuľkách**. Najmä použité hyperparametre, porovnávané štruktúry siete, výsledky jednotlivých spustení, porovnanie dropout-u a porovnanie CPU/GPU by mali byť uvedené tabuľkovo, aby bolo porovnanie čitateľné, jednoznačné a malo primeranú technickú formu. Ukážky minimálneho rozsahu a formy tabuliek, ktoré majú byť súčasťou dokumentácie, sú uvedené na konci dokumentu v Tab. 1 ... Tab. 13.

Odovzdanie a hodnotenie

Hodnotenie zadania prebieha formou osobnej konzultácie. Zadanie sa odovzdáva vždy na **začiatku cvičenia** po odprezentovaní nasledujúceho zadania.

Počas semestra môže študent nahradiť najviac dve cvičenia. V takom prípade je možné zadanie odovzdať aj na cvičení, na ktorom študent nahrádza. V ostatných prípadoch sa zadanie odovzdáva vždy svojmu cvičiacemu.

Všetky požadované výsledky musia byť pripravené **už na začiatku cvičenia** a uvedené v dokumentácii. Opätovné spúšťanie trénovania počas obhajoby sa vykonáva len v prípade, že o to cvičiaci výslovne požiada.

Na vypracovanie tohto zadania sú vyhradené **2 týždne**.

Dokumentácia výsledkov (PDF)

Pripravte stručnú správu (PDF), ktorá obsahuje grafy z viacerých spustení a krátku analýzu vplyvu zvolených parametrov. Pri neurónových sieťach typicky uvádzajte priebeh učenia ako **loss vs. epoch** a podľa typu úlohy aj vhodné metriky. Dokument slúži ako podklad pre diskusiu, aby nebolo nutné čakať na opakované výpočty počas obhajoby.
Pre toto zadanie uveďte v správe:

1. stručný opis úlohy, použitých dát,
2. opis aspoň **2 porovnávaných štruktúr MLP** a použitých hyperparametrov + opis aspoň **3 porovnávaných architektúr CNN** a použitých hyperparametrov,
3. pre **každú** porovnávanú štruktúru MLP aj CNN výsledky **5 nezávislých spustení trénovania odznova**, vrátane minimálnej, maximálnej a priemernej úspešnosti klasifikácie,
4. opis a porovnanie **aspoň 3 ďalších modelov s rôznym dropout-om**, vrátane stručného zhodnotenia, ako sa zmena dropout-u prejavila na pretrénovaní,
5. pre **najlepší výsledok z každej štruktúry** uveďte priebeh učenia, kontingenčnú maticu a dosiahnuté úspešnosti na trénovacích a testovacích dátach,
6. porovnanie **MLP a CNN**,
7. porovnanie času trénovania na **CPU a GPU** aspoň pre jednu vybranú MLP sieť a jednu vybranú CNN sieť,
8. postup testovania vybraných vzoriek a ich **vizuálnu prezentáciu** s uvedením skutočnej triedy, predikovanej triedy a výstupu siete.

Súčasťou dokumentácie **musia byť aj prehľadné tabuľky** so zvolenými hyperparametrami,porovnávanými štruktúrami, výsledkami jednotlivých behov a porovnaním nastavenej hodnoty dropout-u. Všetky priebehy učenia, porovnania a výsledné vizualizácie majú byť uvedené ako **vlastné grafy alebo figúry** s primeraným popisom osí, legendou a označením. **Screenshoty z TensorBoardu alebo z iných nástrojov sa nepovažujú za dostatočnú formu spracovania výsledkov.**

Ukážky minimálneho rozsahu a formy tabuliek sú uvedené na konci dokumentu v Tab. 1, Tab. 2, Tab. 3, Tab. 4, Tab. 5, Tab. 6, Tab. 7, Tab. 8, Tab. 9, Tab. 10, Tab. 11, Tab. 12 a Tab. 13.

Obhajoba riešenia

Súčasťou hodnotenia je demonštrácia funkčného kódu a vysvetlenie implementácie. Body sa udeľujú najmä za porozumenie princípu algoritmu a schopnosť zdôvodniť dosiahnuté výsledky, nie iba za spustiteľný program.

V diskusii buďte pripravení interpretovať grafy učenia (**loss vs. epoch** a prípadné metriky) a zdôvodniť vplyv nastavení (napr. architektúra, regularizácia, learning rate, batch size) na konvergenciu a opakovateľnosť výsledkov.

Pri obhajobe vysvetlite, aké vstupné a výstupné dáta boli použité, aká bola zvolená štruktúra MLP siete, aké architektúry CNN boli porovnávané a aké parametre trénovania boli nastavené. Na základe grafov učenia, chýb a kontingenčných matíc zdôvodnite, prečo bol vybraný najlepší model a aké rozdiely sa prejavili medzi MLP a CNN.

Zároveň vysvetlite, prečo sa pri porovnávaní modelov používa **5 nezávislých spustení trénovania odznova**, ako sa prejavil **dropout** na pretrénovaní a aký rozdiel sa ukázal pri trénovaní na **CPU a GPU**.

Súčasťou obhajoby je aj preukázanie porozumenia vlastnému kódu v **Matlabe aj Pythone**. **AI slop** alebo iný cudzí kód sa považuje za **plagiát**.

Odporúčané nástroje a technická realizácia

Zadanie je možné vypracovať v prostredí **Matlab** alebo v jazyku **Python**. Ako alternatívu bez inštalácie je možné použiť **MATLAB Online** (`https://matlab.mathworks.com/`). Pre Python odporúčame **Google Colab** (`https://colab.research.google.com/`).

Matlab

Pri Matlabe použite vhodné nástroje pre neurónové siete podľa pokynov na cvičení. Do správy zahrňte priebeh učenia (loss vs. epoch) a výsledné metriky podľa zadania.
Pri Matlabe použite dataset `MNIST_MATLAB.zip` z repozitára **UMINT-UNS_data**. Pre MLP aj CNN časť vychádzajte z príslušných ukážkových kódov a tie ďalej upravujte pre potreby tejto úlohy.

Python

Pri Pythone odporúčame pracovať v prostredí Google Colab. Pre úlohy v tomto bloku použite oficiálne repozitáre:

- **Dodatočné kódy a zadania:** `https://github.com/STU-FEI-OUI/UMINT-UNS`
- **Dáta pre Python:** `https://github.com/STU-FEI-OUI/UMINT-UNS_data`

Pre vizualizáciu uveďte aspoň priebeh učenia ako **loss vs. epoch** a podľa zadania doplňte validáciu a metriky.

Pri Pythone použite oficiálny dataset `MNIST` z knižnice `torchvision`. Dokumentácia k datasetu je dostupná na: `https://docs.pytorch.org/vision/stable/generated/torchvision.datasets.MNIST.html`
Ukážka načítania tréningového a testovacieho datasetu:

```
from torchvision import datasets, transforms
transform = transforms.ToTensor()
train_dataset = datasets.MNIST(root="./data", train=True, download=True, transform=transform)
test_dataset = datasets.MNIST(root="./data", train=False, download=True, transform=transform)
```

V tomto prípade parameter `train=True` načíta **tréningový dataset** a parameter `train=False` **testovací dataset**. Tréningové dáta potom podľa potreby ďalej rozdeľte na **trénovaciu a validačnú množinu**, testovací dataset ponechajte oddelený pre finálne vyhodnotenie.
Ukážka vytvorenia `DataLoader`-ov:

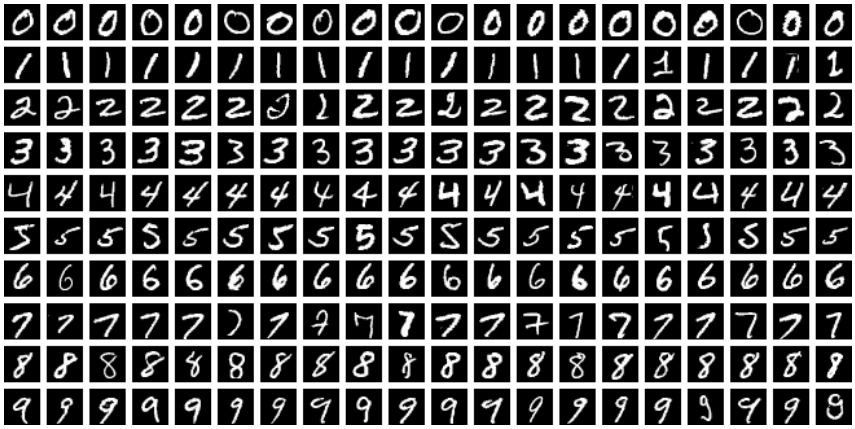
```
from torch.utils.data import DataLoader
train_loader = DataLoader(train_dataset, batch_size=64, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=64, shuffle=False)
```

V prípade potreby ďalších informácií o použití datasetu `MNIST` alebo o iných dostupných datasetoch v PyTorchi pozrite dokumentáciu `torchvision.datasets` `https://docs.pytorch.org/vision/stable/datasets.html`.

Teoretické poznámky

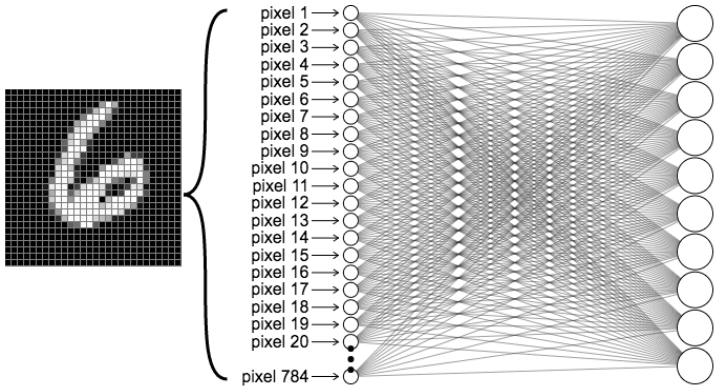
MNIST a základné princípy rozpoznávania číslic

Dataset **MNIST** patrí medzi klasické ukážkové datasety pre úlohy klasifikácie obrazu. Obsahuje rukou písané číslice od **0** do **9**, pričom každá vzorka predstavuje jeden obrázok číslice. Ukážka typických vzoriek z datasetu je na Obr. 1. Na prvý pohľad je vidieť, že jednotlivé triedy síce reprezentujú tie isté číslice, ale ich tvar sa medzi vzorkami mení. Práve táto variabilita je dôvod, prečo nestačí jednoduché porovnanie pixel po pixeli a je potrebné použiť model, ktorý sa naučí všeobecnejšie rozlišovacie znaky.



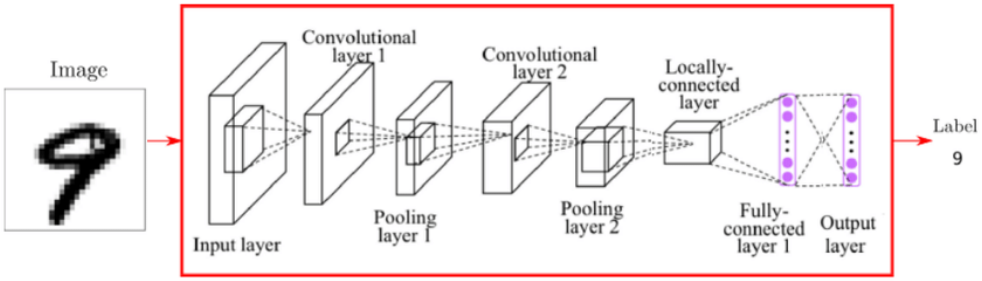
Obr. 1: Ukážka vzoriek z datasetu MNIST.

Pri **MLP** prístupe sa obrázok najprv prevedie na jednorozmerný vstup a potom sa spracúva v plne prepojenej sieti. Takýto model pracuje s číselnou reprezentáciou vstupu, ale explicitne nezachováva priestorové susednosti medzi pixelmi. Princíp klasifikácie číslic pomocou MLP siete je schematicky znázornený na Obr. 2. Výhodou tohto prístupu je jednoduchosť implementácie a architektúry, nevýhodou je, že sieť nevyužíva obrazovú štruktúru vstupu tak prirodzene ako CNN.



Obr. 2: Principiálna schéma rozpoznávania rukou písaných číslic pomocou MLP siete.

Pri **CNN** prístupe sa vstup spracúva ako obraz, teda zachováva sa jeho dvojrozmerná štruktúra. Sieť potom využíva konvolučné vrstvy na hľadanie lokálnych príznakov, napríklad hrán, oblúkov alebo jednoduchých tvarov, ktoré sa v ďalších vrstvách kombinujú do zložitejších vzorov. Typická princípová schéma takéhoto spracovania je na Obr. 3. Práve preto býva CNN pri obrazových úlohách prirodzenejším riešením než klasická MLP sieť.



Obr. 3: Principiálna schéma rozpoznávania rukou písaných číslic pomocou CNN siete.

Zmyslom tohto zadania preto nie je len dosiahnuť čo najvyššiu úspešnosť klasifikácie, ale aj pochopiť rozdiel medzi tým, ako ten istý problém rieši **klasická plne prepojená sieť** a ako ho rieši **konvolučná sieť**. Na ďalších podsekciách potom nadväzuje podrobnejšie vysvetlenie reprezentácie vstupu, delenia dát, opakovaných behov a férového porovnávania architektúr.

MLP vs. CNN a reprezentácia vstupu

V tomto zadaní sa porovnávajú dva odlišné prístupy k rozpoznávaniu obrázkov. Pri **MLP** sieti sa obraz spracúva ako obyčajný vektor hodnôt, teda bez explicitnej informácie o tom, ktoré pixely spolu v obraze susedia. Pri **CNN** sieti sa naopak zachováva obrazová štruktúra vstupu, takže sieť môže pracovať s lokálnymi vzormi a priestorovými vzťahmi medzi pixelmi.

Pri CNN časti sa vstup ponecháva v obrazovom tvare **28×28×1**. Posledná dimenzia predstavuje počet kanálov obrazu. V prípade datasetu MNIST ide o **jednokanálový obraz** v odtieňoch sivej. Pri farebných obrázkoch by táto dimenzia bola väčšia, napríklad **3** pre RGB obraz alebo **4** pri obraze s alfa kanálom.

Pri MLP časti sa naopak obraz nepoužíva v pôvodnom obrazovom tvare, ale prechádza do jednorozmernej reprezentácie. Práve preto je pri plne prepojenej sieti potrebné uvažovať o vstupnej dimenzii inak než pri CNN.

Z praktického hľadiska sa pri MLP očakáva jednoduchšia architektúra a priamočiarejšie spracovanie vstupu, zatiaľ čo pri CNN sa využíva štruktúra obrazu, konvolučné filtre a typicky aj pooling vrstvy. To, že CNN býva pri obrazových dátach prirodzenejším riešením, neznamená, že MLP časť je zbytočná. Naopak, porovnanie oboch prístupov pomáha pochopiť, čo sa zmení, keď sa ten istý problém rieši ako klasický ANN model a ako sieť určená priamo na prácu s obrazom. `https://ujjwalkarn.me/2016/08/11/intuitive-explanation-convnets/`

Rozdelenie dát

Dataset MNIST je už štandardne rozdelený na **tréningovú** a **testovaciu** časť. Testovacia časť má zostať oddelená na finálne vyhodnotenie modelu. Validačná množina sa vytvára ďalším rozdelením tréningových dát. Na tieto zásady nadväzujú poznámky z predchádzajúcich zadaní o delení datasetu na trénovacie, validačné a testovacie dáta.

Férové porovnávanie architektúr

Pri porovnávaní rôznych štruktúr sietí je potrebné zachovať čo najviac **rovnaké hlavné hyperparametre** trénovania, aby bolo možné výsledky medzi modelmi férovo porovnať. Inak by nebolo jasné, či rozdiel vo výsledku spôsobila samotná architektúra alebo zmena nastavenia učenia.

Pri MLP sa tak porovnávajú najmä rôzne počty neurónov a vrstiev, pri CNN napríklad počet konvolučných vrstiev, filtrov alebo ďalšie architektonické prvky. V tomto zadaní sa navyše sleduje aj vplyv **dropout-u**, teda regularizácie, ktorá môže obmedziť pretrénovanie, ale pri nevhodnom nastavení môže zhoršiť samotné učenie.

Ako interpretovať výsledky pri klasifikácii obrázkov

Pri klasifikácii obrázkov nestačí sledovať iba **accuracy**. Počas trénovania je dôležitý aj priebeh **loss vs. epoch**, prípadne rozdiel medzi trénovacou a validačnou chybou. Pri porovnaní MLP a CNN má význam sledovať nielen výslednú úspešnosť, ale aj stabilitu medzi behmi, správanie pri dropout-e a známky pretrénovania.

Pri interpretácii výsledkov je užitočné využiť aj **kontingenčnú maticu**, ktorá ukáže, medzi ktorými triedami sa model najčastejšie mýli. V tomto zadaní má navyše význam sledovať aj rozdiel medzi tréningom na **CPU** a **GPU**, pretože okrem kvality modelu zohráva úlohu aj výpočtová náročnosť.

Ukážky minimálneho rozsahu tabuliek v dokumentácii

Model	Skryté vrstvy	Neuróny	Epochy	LR	Batch
MLP1	1	128	20	0.001	64
MLP2	2	256,128	20	0.001	64

Tabuľka 1: Porovnávané štruktúry MLP a hlavné hyperparametre.

Beh	Train loss	Test loss	Train acc [%]	Test acc [%]
1	0.061	0.089	98.2	97.3
2	0.057	0.094	98.4	97.0
3	0.063	0.086	98.1	97.5
4	0.059	0.091	98.3	97.2
5	0.060	0.088	98.2	97.4

Tabuľka 2: Výsledky 5 spustení pre prvú štruktúru MLP. Rovnaká tabuľka má byť uvedená pre každú porovnávanú štruktúru, teda minimálne 2 tabuľky pre MLP časť.

Beh	Train loss	Test loss	Train acc [%]	Test acc [%]
1	0.052	0.081	98.6	97.6
2	0.049	0.084	98.8	97.4
3	0.055	0.079	98.5	97.8
4	0.051	0.082	98.7	97.5
5	0.050	0.080	98.7	97.7

Tabuľka 3: Výsledky 5 spustení pre druhú štruktúru MLP.

Model	Min test acc [%]	Max test acc [%]	Priemer test acc [%]	Priemer test loss
MLP1	96.8	97.4	97.1	0.093
MLP2	97.4	97.8	97.6	0.081

Tabuľka 4: Súhrnné porovnanie porovnávaných štruktúr MLP.

Model	Conv vrstvy	Filtre	FC vrstvy	Epochy	Batch
CNN1	2	32,64	128	15	64
CNN2	3	32,64,128	128	15	64
CNN3	3	32,64,128	256	15	64

Tabuľka 5: Porovnávané architektúry CNN a hlavné hyperparametre.

Beh	Train loss	Test loss	Train acc [%]	Test acc [%]
1	0.024	0.038	99.3	98.9
2	0.026	0.041	99.2	98.8
3	0.022	0.036	99.4	99.0
4	0.023	0.039	99.3	98.9
5	0.025	0.040	99.2	98.8

Tabuľka 6: Výsledky 5 spustení pre prvú architektúru CNN. Rovnaká tabuľka má byť uvedená pre každú porovnávanú architektúru, teda minimálne 3 tabuľky pre CNN časť.

Beh	Train loss	Test loss	Train acc [%]	Test acc [%]
1	0.018	0.029	99.6	99.1
2	0.021	0.032	99.5	99.0
3	0.017	0.028	99.7	99.2
4	0.019	0.031	99.6	99.1
5	0.020	0.030	99.5	99.0

Tabuľka 7: Výsledky 5 spustení pre druhú architektúru CNN.

Beh	Train loss	Test loss	Train acc [%]	Test acc [%]
1	0.015	0.026	99.7	99.3
2	0.017	0.028	99.6	99.2
3	0.014	0.025	99.8	99.4
4	0.016	0.027	99.7	99.3
5	0.015	0.026	99.7	99.3

Tabuľka 8: Výsledky 5 spustení pre tretiu architektúru CNN.

Model	Min test acc [%]	Max test acc [%]	Priemer test acc [%]	Priemer test loss
CNN1	98.8	99.0	98.9	0.039
CNN2	99.0	99.2	99.1	0.030
CNN3	99.2	99.4	99.3	0.026

Tabuľka 9: Súhrnné porovnanie porovnávaných architektúr CNN.

Model	Dropout	Počet behov	Priemer epochy začiatku pretrénovania	Priemer val loss
D1	0.0	5	7.8	0.054
D2	0.3	5	11.6	0.037
D3	0.5	5	14.2	0.041

Tabuľka 10: Porovnanie modelov s rôznym nastavením dropout-u.

Dropout	Počet behov	Priemer train acc [%]	Priemer test acc [%]	Priemer test loss
0.0	5	99.7	98.8	0.041
0.3	5	99.4	99.1	0.029
0.5	5	99.0	98.9	0.034

Tabuľka 11: Súhrnné porovnanie priemerných výsledkov pre rôzne nastavenia dropout-u.

Prístup	Počet modelov	Priemer test acc [%]	Priemer test loss
MLP	2	97.6	0.081
CNN	3	99.1	0.031

Tabuľka 12: Súhrnné porovnanie priemerných výsledkov MLP a CNN.

Model	**CPU čas [s]**	**GPU čas [s]**	**Zrýchlenie**
MLP2	148.2	24.7	6.0×
CNN2	512.6	63.9	8.0×
Priemer	330.4	44.3	7.5×

Tabuľka 13: Súhrnné porovnanie času trénovania na CPU a GPU.